Lesson 11: Dependencies

Learning Objectives

- Understand implicit vs explicit dependencies
- Use `depends_on` to declare explicit dependencies
- Understand how Terraform builds the dependency graph
- Use `terraform graph` to visualize dependencies

---

1. Implicit Dependencies (Automatic)

Terraform automatically detects dependencies when one resource references an attribute of another:

```
resource "aws_s3_bucket" "data" {
  bucket = "my-data-bucket"
}

# Implicit dependency: aws_s3_bucket_versioning depends on aws_s3_bucket
resource "aws_s3_bucket_versioning" "data" {
  bucket = aws_s3_bucket.data.id  # <-- reference creates dependency

  versioning_configuration {
    status = "Enabled"
  }
}
```

Terraform reads `aws_s3_bucket.data.id` and knows it must create the bucket **before** the versioning config.

How it works

```
Terraform analyzes your config and builds a graph:

aws_s3_bucket.data ──► aws_s3_bucket_versioning.data
                                    │
                                    ▼
                       aws_s3_bucket_public_access_block.data
```

References can be:

- Direct: `aws_s3_bucket.data.id`
- Nested: `aws_s3_bucket.data.arn`
- Through locals: `local.bucket_id` (if local references a resource)
- Through modules: `module.storage.bucket_id`

---

2. Explicit Dependencies (`depends_on`)

Use `depends_on` when there is **no direct reference** but you need to enforce ordering:

```
resource "aws_s3_bucket" "data" {
  bucket = "my-data-bucket"
}

resource "aws_sqs_queue" "notifications" {
  name = "notifications-queue"

  depends_on = [aws_s3_bucket.data]  # <-- explicit dependency
}
```

When to use `depends_on`

Scenario	Example
IAM policy applied before dependent resource	Queue must exist before policy references it
Cross-service ordering with no direct reference	Bucket must be created before a Lambda that reads it
Side effects	A resource triggers a process, another resource must wait

`depends_on` Syntax

```
depends_on = [
  aws_s3_bucket.data,
  aws_dynamodb_table.items,
  module.iam,
  aws_sqs_queue.main,
]
```

Each entry is a **resource address**: `resource_type.local_name` or `module.module_name`.

---

3. Dependency Graph Visualization

Terraform can generate a dependency graph in DOT format:

```
terraform graph
```

Install Graphviz to render it:

```
# macOS
brew install graphviz

# Linux
sudo apt install graphviz
```

Render to PNG:

```
terraform graph | dot -Tpng > graph.png
open graph.png
```

Or render to SVG for browser viewing:

```
terraform graph | dot -Tsvg > graph.svg
open graph.svg
```

Example Graph Output

```
digraph {
  compound = "true"
  newrank = "true"
  subgraph "root" {
    "[root] aws_s3_bucket.data" [label = "aws_s3_bucket.data", shape =
"box"]
    "[root] aws_s3_bucket_versioning.data" [label =
"aws_s3_bucket_versioning.data", shape = "box"]
    "[root] aws_sqs_queue.notifications" [label =
"aws_sqs_queue.notifications", shape = "box"]
    "[root] aws_s3_bucket.data" -> "[root] aws_s3_bucket_versioning.data"
    "[root] aws_s3_bucket.data" -> "[root] aws_sqs_queue.notifications"
  }
}
```

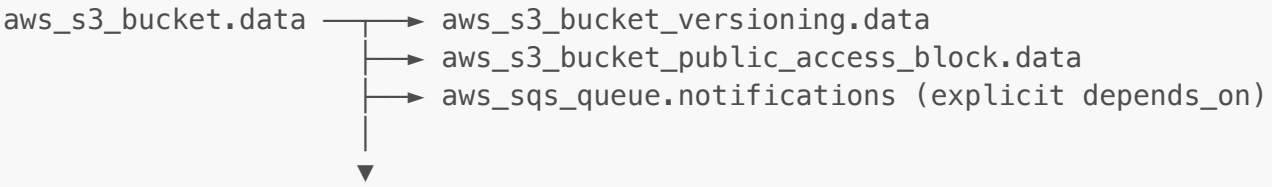
---

4. Complete Example with Mixed Dependencies

```
# provider.tf
provider "aws" {
  # ... LocalStack config
}

# main.tf
locals {
```

```
  name_prefix = "dep-demo"
}

# No dependencies (root-level resource)
resource "aws_s3_bucket" "data" {
  bucket = "${local.name_prefix}-data"
}

# Implicit dependency — references aws_s3_bucket.data.id
resource "aws_s3_bucket_versioning" "data" {
  bucket = aws_s3_bucket.data.id
  versioning_configuration {
    status = "Enabled"
  }
}

# Implicit dependency — references aws_s3_bucket.data.arn
resource "aws_s3_bucket_public_access_block" "data" {
  bucket = aws_s3_bucket.data.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

# Has no implicit dependency on S3, but we want ordering
resource "aws_sqs_queue" "notifications" {
  name = "${local.name_prefix}-notifications"

  depends_on = [aws_s3_bucket.data]
}

# Depends on both queue and bucket
resource "aws_dynamodb_table" "items" {
  name         = "${local.name_prefix}-items"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "item_id"

  attribute {
    name = "item_id"
    type = "S"
  }

  depends_on = [
    aws_sqs_queue.notifications,
    aws_s3_bucket.data,
  ]
}
```

Dependency Graph



---

5. Circular Dependencies

Terraform prevents circular dependencies. This **will not work**:

```
# Error: Cycle error
resource A {
  depends_on = [B]
}

resource B {
  depends_on = [A]
}
```

Fix by restructuring: introduce a third resource or remove one of the dependencies.

---

6. Dependency Testing

```
# See the dependency graph as text
terraform graph

# See the execution plan order
terraform plan

# See what depends on what
terraform state list
```

In the plan output, resources are listed in dependency order:

```
Terraform will perform the following actions:

  # aws_s3_bucket.data will be created  (created first)
  # aws_s3_bucket_versioning.data will be created  (second)
  # aws_sqs_queue.notifications will be created  (third)
  # aws_dynamodb_table.items will be created  (last)
```

---

7. Key Takeaways

- **Implicit** dependencies come from resource attribute references — Terraform detects these automatically
- **Explicit** dependencies use `depends_on` — use when there's no reference but ordering matters
- `depends_on` takes a list of resource or module addresses
- `terraform graph` visualizes the dependency graph (requires Graphviz for images)
- Circular dependencies cause an error — restructure to avoid them
- The plan output shows resource creation order based on the dependency graph

example and exercise repo url

https://github.com/eyu-se/terraform-training-material-with-projects

Trainer - Eyuel M.